



ARGUMENTOS

CAPÍTULO 19: ARGUMENTOS

<https://books.goalkicker.com/CBook/>



[https://github.com/rafael-vieira-coelho/LinguagemC/
tree/Aula-11---Argumentos](https://github.com/rafael-vieira-coelho/LinguagemC/tree/Aula-11---Argumentos)

» Enquanto isto, na aula passada....

escreve_arquivo_binario.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  typedef struct jogador {
5      int    identificador;
6      char   nome[30];
7      float  pontos;
8      int    ranking;
9  }Jogador;
10
11 int main() {
12     FILE *arquivo;
13     arquivo = fopen("exemplo.bin", "rwb");
14     if (arquivo != NULL) {
15         Jogador player;
16         //Escrita das structs Jogador
17         player.identificador = 1;
18         strcpy(player.nome, "Jorge da Silva");
19         player.pontos = 987.21;
20         player.ranking = 12;
21         fwrite(&player, sizeof(Jogador), 1, arquivo);
```

Como Receber Dados sem Leitura?

Parâmetro	Detalhes
argc	Contagem de argumentos (<i>argument count</i>) – inicializado com o número de argumentos separados por espaços passados ao programa a partir da linha de comandos, bem como o próprio nome do programa.
argv	Vetor de argumentos (<i>argument vector</i>) – inicializado como um <i>array</i> de ponteiros para caracteres (<i>strings</i>) contendo os argumentos (e o nome do programa) que foram fornecidos na linha de comandos.

Como Implementar em C?

» Recebemos como parâmetro no método principal.

```
ex1.c ×  
1 #include <stdio.h>  
2  
3 ▶ int main(int argc, char *argv[]) {  
4     // ...  
5     return 0;  
6 }
```

Remodelando o Exemplo Antigo

escrita_args.c

```
12 ▶ int main(int argc, char *argv[]) {
13     // Validação dos argumentos necessários (Nome do programa + 5 parâmetros)
14     if (argc != 6) {
15         fprintf(stderr, "Uso: %s <arquivo.bin> <id> <nome> <pontos> <ranking>\n", argv[0]);
16         return EXIT_FAILURE;
17     }
18     // Abre o ficheiro para escrita e leitura binária ("w+b" cria ou sobreescreve)
19     FILE *arquivo = fopen(filename:argv[1], mode:"w+b");
20     if (arquivo != NULL) {
21         Jogador player;
22         // Atribuição dos dados passados via linha de comandos
23         player.identificador = atoi(argv[2]);
24         strncpy(player.nome, argv[3], sizeof(player.nome) - 1);
25         player.nome[sizeof(player.nome) - 1] = '\0'; // Garante terminação nula
26         player.pontos = atof(argv[4]);
27         player.ranking = atoi(argv[5]);
28
29         // Escrita da struct Jogador no ficheiro
30         fwrite(&player, size:sizeof(Jogador), nitems:1, arquivo);
```

```
coelho@mac exemplos % gcc escrita_args.c -o escrita.o
```

```
coelho@mac exemplos % ./escrita.o arquivo.bin 1 Joe 10 2  
ID: 1 | NOME: Joe | Pontos: 10.00 | Ranking: 2
```

escrita_args.c ×

```
12 ▶ int main(int argc, char *argv[]) {  
13     // Validação dos argumentos necessários (Nome do programa + 5 parâmetros)  
14     if (argc != 6) {  
15         fprintf(stderr, "Uso: %s <arquivo.bin> <id> <nome> <pontos> <ranking>\n", argv[0]);  
16         return EXIT_FAILURE;  
17     }  
18     // Abre o ficheiro para escrita e leitura binária ("w+b" cria ou sobrescreve)  
19     FILE *arquivo = fopen(filename:argv[1], mode:"w+b");  
20     if (arquivo != NULL) {  
21         Jogador player;  
22         // Atribuição dos dados passados via linha de comandos  
23         player.identificador = atoi(argv[2]);  
24         strncpy(player.nome, argv[3], sizeof(player.nome) - 1);  
25         player.nome[sizeof(player.nome) - 1] = '\0'; // Garante terminação nula  
26         player.pontos = atof(argv[4]);  
27         player.ranking = atoi(argv[5]);  
28  
29         // Escrita da struct Jogador no ficheiro  
30         fwrite(&player, size:sizeof(Jogador), nitems:1, arquivo);
```



OBRIGADO!

RAFAELVC2@GMAIL.COM

<https://books.goalkicker.com>